Model Evaluation

evaluating the ML detector (Isolation Forest) on a labeled dataset of 109 sessions, the model is trained on all sessions then scores each one to measure detection performance

1. Parse and label sessions

parsing the anonymized log file to extract sessions and features, a session is labeled normal if it logged in as deploy_user, otherwise it is labeled attack

```
import pandas as pd
from ftp_ids.parsers.vsftpd_parser import VsftpdParser
from ftp_ids.core.session_builder import build_sessions
from ftp_ids.core.feature_extractor import FeatureExtractor

parser = VsftpdParser()
fe = FeatureExtractor()

with open("../data/samples/vsftpd.log") as f:
    events = [parser.parse_line(line) for line in f if line.strip()]

sessions = build_sessions([e for e in events if e])

df = pd.DataFrame([{"user": s["user"], **fe.extract(s)} for s in sessions])
df["label"] = "attack"
df.loc[(df["user"] == "deploy_user") & (df["is_auth"] == 1), "label"] = "normal"

df["label"].value_counts()
```

```
label
normal    98
attack    11
Name: count, dtype: int64
```

2. Train the model

the model trains on all sessions, Isolation Forest uses the contamination parameter (0.05) to expect a small proportion of anomalies in the training data, I used 5% to reflect my real dataset anomaly count

```
from ftp_ids.core.detector import Detector
from ftp_ids.core.feature_extractor import FEATURE_NAMES

detector = Detector(model_path="/tmp/eval_model.pkl", contamination=0.05)
detector.train_on_features(df[FEATURE_NAMES])
```

```
Model retrained and saved to /tmp/eval_model.pkl
```

3. Score all sessions

each session gets a score from 0.0 to 1.0, a low score means the session looks normal, a high score means it looks suspicious

```
df["score"] = detector.score_batch(sessions)

df.groupby("label")["score"].describe().round(3)
```

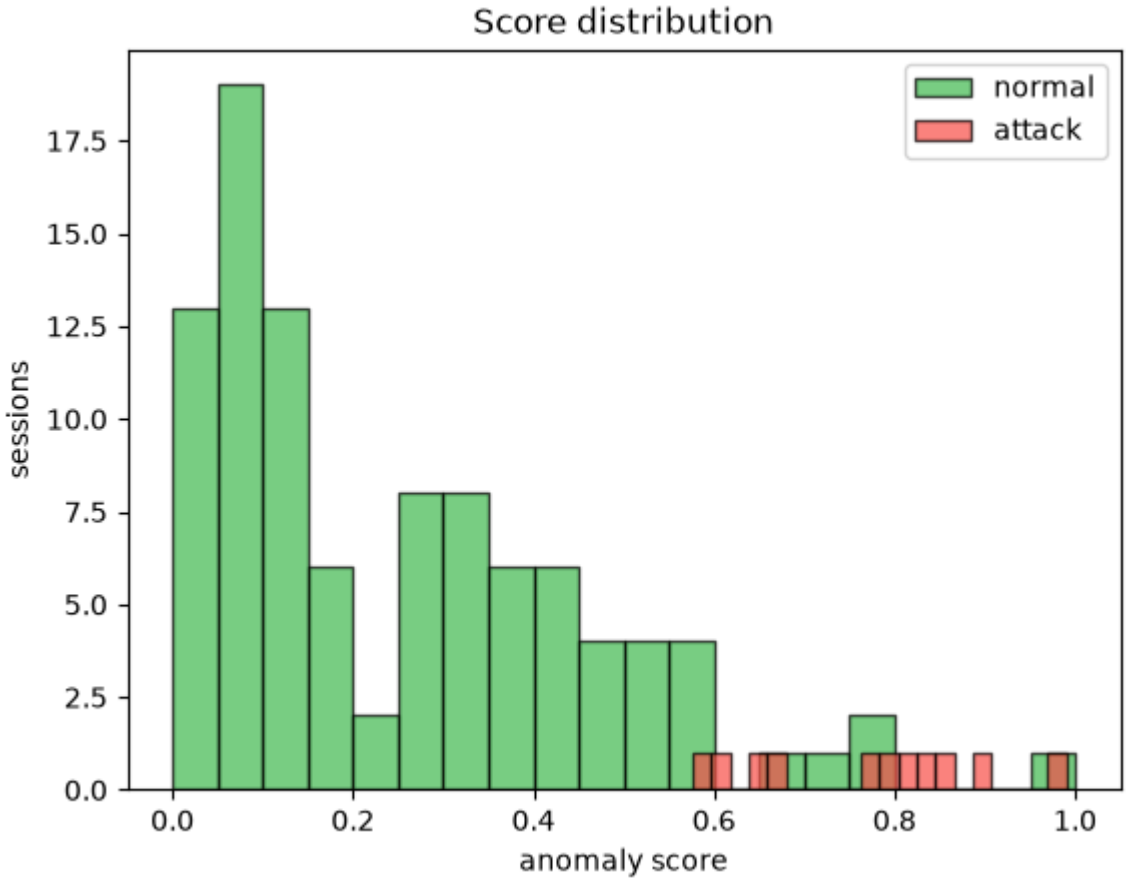
label	count	mean	std	min	25%	50%	75%	max
attack	11.0	0.769	0.129	0.577	0.65	0.803	0.842	0.99
normal	98.0	0.253	0.212	0.000	0.08	0.186	0.388	1.00

4. Score distribution

this chart shows how normal and attack sessions are scored, if the two groups separate clearly the model can tell them apart

```
import matplotlib.pyplot as plt

plt.hist(df[df["label"] == "normal"]["score"], bins=20, alpha=0.7, color="#3fb950", edgecol
plt.hist(df[df["label"] == "attack"]["score"], bins=20, alpha=0.7, color="#f85149", edgecol
plt.title("Score distribution")
plt.xlabel("anomaly score")
plt.ylabel("sessions")
plt.legend()
plt.show()
```



5. Threshold sweep

I tested every threshold from 0.05 to 0.95 to find the one with the best F1 score, the best threshold catches the most attacks without flagging too many normal sessions

```
import numpy as np
from sklearn.metrics import precision_recall_fscore_support

rows = []
for t in np.arange(0.05, 1.0, 0.05).round(2):
    y_pred = np.where(df["score"] >= t, "attack", "normal")
    p, r, f, _ = precision_recall_fscore_support(df["label"], y_pred, pos_label="attack", a
    rows.append({"threshold": t, "precision": p, "recall": r, "f1": f})

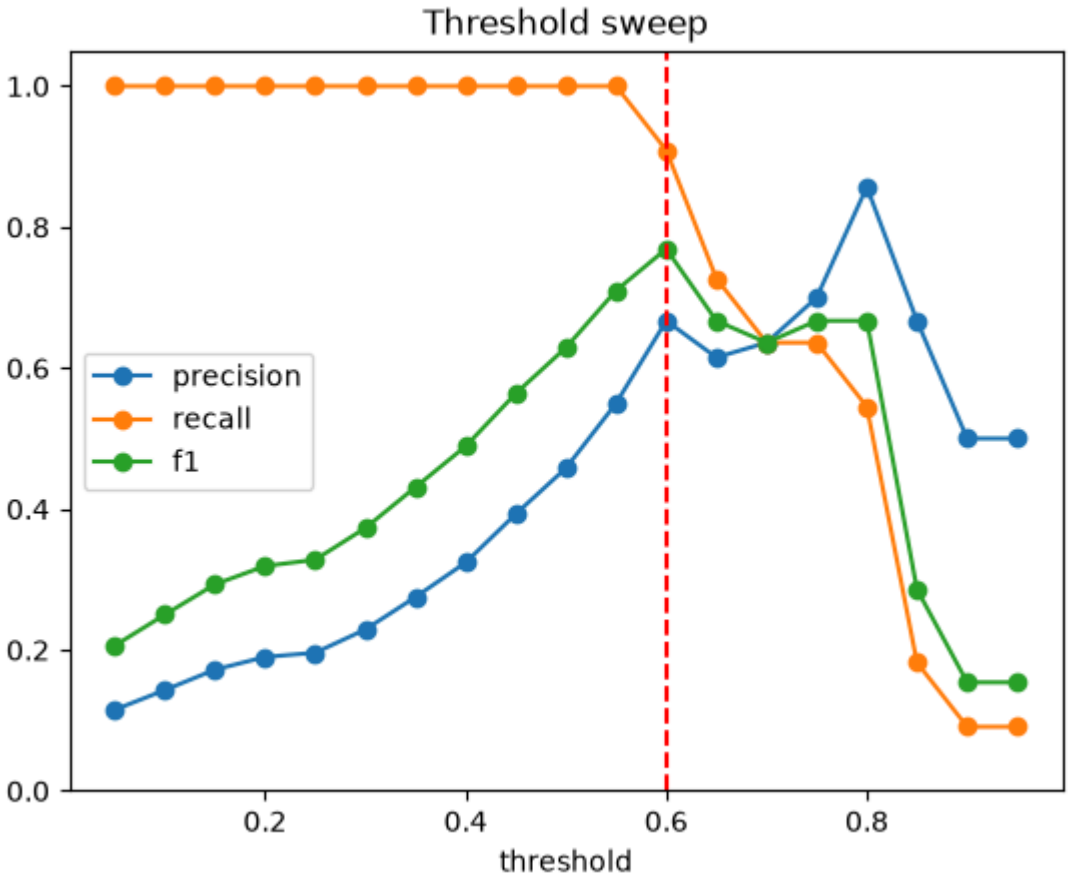
results = pd.DataFrame(rows).round(3)
best = results.loc[results["f1"].idxmax()]

print("best threshold:", best["threshold"], "f1:", best["f1"])
results
```

```
best threshold: 0.6 f1: 0.769
```

	threshold	precision	recall	f1
0	0.05	0.115	1.000	0.206
1	0.10	0.143	1.000	0.250
2	0.15	0.172	1.000	0.293
3	0.20	0.190	1.000	0.319
4	0.25	0.196	1.000	0.328
5	0.30	0.229	1.000	0.373
6	0.35	0.275	1.000	0.431
7	0.40	0.324	1.000	0.489
8	0.45	0.393	1.000	0.564
9	0.50	0.458	1.000	0.629
10	0.55	0.550	1.000	0.710
11	0.60	0.667	0.909	0.769
12	0.65	0.615	0.727	0.667
13	0.70	0.636	0.636	0.636
14	0.75	0.700	0.636	0.667
15	0.80	0.857	0.545	0.667
16	0.85	0.667	0.182	0.286
17	0.90	0.500	0.091	0.154
18	0.95	0.500	0.091	0.154

```
results.plot(x="threshold", y=["precision", "recall", "f1"], marker="o", ylim=(0, 1.05), ti
plt.axvline(best["threshold"], color="red", linestyle="--")
plt.show()
```



6. Confusion matrix

this shows how many sessions the model got right and wrong at the best threshold, rows are what the session really is, columns are what the model predicted

```
import seaborn as sns
from sklearn.metrics import classification_report, confusion_matrix

y_pred = np.where(df["score"] >= best["threshold"], "attack", "normal")

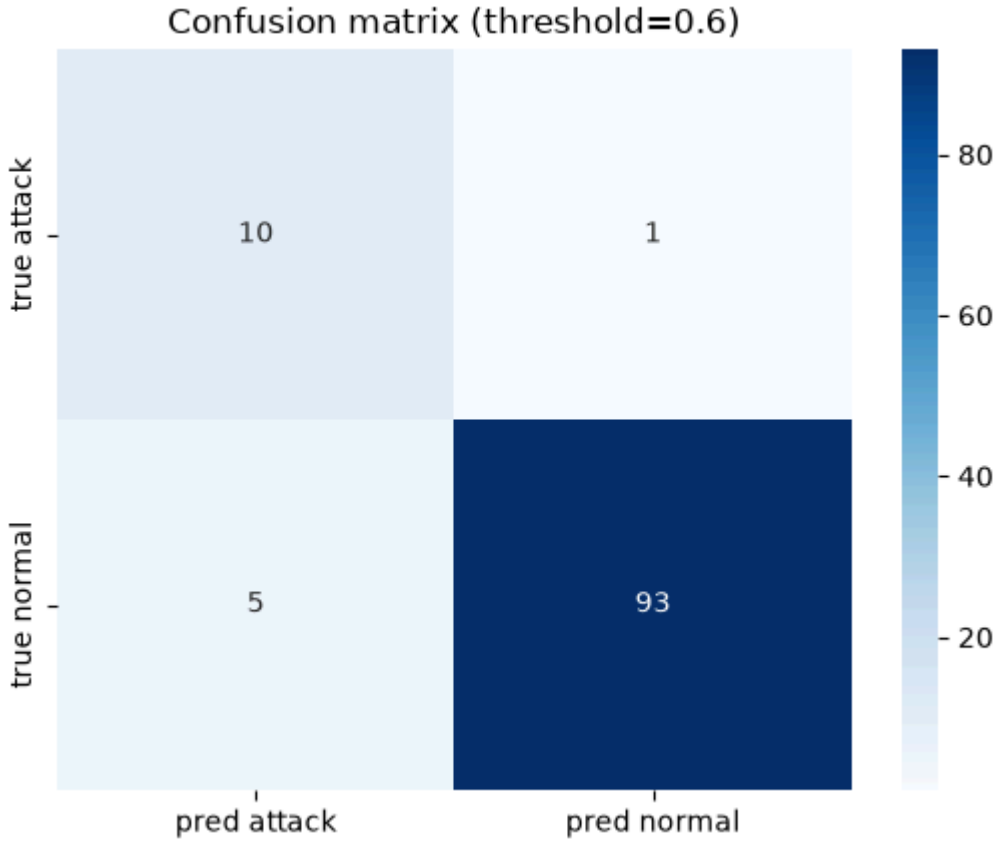
print(classification_report(df["label"], y_pred))

cm = confusion_matrix(df["label"], y_pred, labels=["attack", "normal"])
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues",
            xticklabels=["pred attack", "pred normal"],
            yticklabels=["true attack", "true normal"])
plt.title(f"Confusion matrix (threshold={best['threshold']})")
plt.show()
```

```
              precision    recall  f1-score   support

      attack       0.67      0.91      0.77        11
      normal       0.99      0.95      0.97        98

    accuracy                           0.94       109
   macro avg       0.83      0.93      0.87       109
weighted avg       0.96      0.94      0.95       109
```



7. Summary

```
print("sessions:", len(df), "normal:", (df["label"] == "normal").sum(), "attack:", (df["lab
print("contamination: 0.05")
print("best threshold:", best["threshold"])
print("precision:", best["precision"], "recall:", best["recall"], "f1:", best["f1"])
```

```
sessions: 109 normal: 98 attack: 11
contamination: 0.05
best threshold: 0.6
precision: 0.667 recall: 0.909 f1: 0.769
```